

ReadRight 2.0 - Milestone 2

Initial Prompt Log

AI Systems Used: Claude (Anthropic) and Gemini (Google)

Entry 1: Setting Up the Backend Proxy - server.js Structure (Claude)	
1. Context	We needed to build the thin backend proxy that holds the OpenAI API key server-side so the Flutter app never embeds it. The assignment requires all API calls to be routed through this proxy. I had never written a Node.js Express server before and needed to understand what the minimum viable structure looked like.
2. Prompt Excerpt	"I need to build a thin Express.js backend proxy for a Flutter literacy app. The proxy needs to hold the OpenAI API key server-side and expose one endpoint, which is POST /api/story, that accepts a list of Dolch sight words and a theme, calls GPT-4o-mini, and returns a short children's story. The Flutter app should never see the key. What does the minimum server.js look like?"
3. AI Summary	Claude provided a working Express server structure using .env for key loading, a single POST /api/story route, and a fetch call to the OpenAI chat completions endpoint. It also suggested a GET /health endpoint for quick testing, a CORS middleware setup, and a check that kills the server on startup if OPENAI_API_KEY is missing from the environment.
4. Human Evaluation	The structure was solid and matched what the assignment required. The key-on-startup-check was a good catch, without it the server starts silently and fails only at request time, which is harder to debug. I did not use Claude's suggested error message wording verbatim. The prompt-building logic it sketched was too generic and didn't account for Dolch word constraints or grade level, so I rewrote that part myself.
5. Final Decision	Accepted the overall structure and the health-check / startup-guard pattern. Rejected Claude's prompt-building function and rewrote it with a system prompt specifically about Dolch sight words, grade level, and a 3–5 sentence story limit to match the ReadRight literacy context. The final buildDolchStoryPrompt() in server.js is entirely our own work.
6. Testing / Verification	Started the server with node server.js and hit GET /health, got. Then tested POST /api/story with curl using the sample Dolch words ['all', 'am', 'are'] and confirmed a valid story came back. Also verified the server exits immediately with a clear error if OPENAI_API_KEY is removed from backend/.env.

Entry 2: Android Emulator URL for Localhost Proxy (Gemini)	
1. Context	The StoryService in the Flutter app needs to hit http://localhost:8787 on the backend proxy. It worked fine on Flutter Web (Chrome) but when we ran on the

	Android emulator, the story generation failed with a connection refused error. I needed to understand why and how to fix it.
2. Prompt Excerpt	"My Flutter app calls http://localhost:8787 to reach a local Express backend proxy. It works in the browser but fails on the Android emulator with a connection error. Why does localhost not work on Android emulators, and what is the correct host address to use?"
3. AI Summary	Gemini explained that the Android emulator runs as a separate virtual machine and its loopback interface (localhost / 127.0.0.1) refers to the emulator itself, not the host machine running the Mac. The Android emulator reserves the address 10.0.2.2 as a special alias that always routes to the host machine's localhost. So the Flutter app should use http://10.0.2.2:8787 when running on Android.
4. Human Evaluation	This was correct and matched what we found in Android's official emulator networking documentation. It is a well-known Android development fact, Gemini did not add anything beyond the standard explanation. The platform detection logic needed to check kIsWeb first and then Platform.isAndroid, which Gemini also sketched but we refined.
5. Final Decision	Accepted. We implemented platform-based URL selection in story_service.dart: kIsWeb returns localhost:8787, Platform.isAndroid returns 10.0.2.2:8787, and all other platforms (iOS simulator, macOS desktop) return localhost:8787. We verified the logic by reading Dart's Platform documentation to confirm Platform.isAndroid is safe to call after the kIsWeb guard.
6. Testing / Verification	Ran flutter run on the Android emulator, tapped the Generate Test Story button in the StorySpikeScreen, and received a valid story back from the proxy. Also confirmed the web build still worked correctly with flutter run -d chrome. Checked the story_service.dart _baseUrl getter against the live behavior to ensure the platform branching matched what we expected.

Entry 3: Designing the Dolch Story Prompt for Claude	
1. Context	We needed to write the system and user prompts that get sent to GPT-4o-mini from the backend proxy. The story has to be kindergarten-appropriate, short (3–5 sentences), and must include the specific Dolch sight words provided by the teacher. I wanted to understand what prompt structure would reliably produce that kind of output and avoid the model wandering into complex vocabulary.
2. Prompt Excerpt	"I am building a children's literacy app. I need Claude to generate a 3-5 sentence kindergarten story that naturally uses a given list of Dolch sight words. The model keeps writing longer stories or ignoring the word list. What system prompt and user prompt structure will reliably constrain it to short stories that actually include the target words?"
3. AI Summary	Claude recommended separating responsibilities between the system prompt and the user prompt. The system prompt should define the persona (children's story writer), the hard constraints (always exactly 3–5 sentences, no title, no headings),

	and the grade-level framing. The user prompt should provide the specific word list and theme per call. It also suggested capping max_tokens at around 300 to act as a hard length ceiling on the API side.
4. Human Evaluation	The separation of system vs. user prompt responsibility was the most useful insight, and it keeps the constraints stable across calls while allowing the word list and theme to vary per request. The max_tokens=300 ceiling was a good practical guardrail. Claude's suggested persona wording was generic and I rewrote it to be more specific to ReadRight. The 'no title, no headings' instruction was our own addition after early test outputs included headers.
5. Final Decision	Accepted the system/user split and the max_tokens guardrail. Rewrote both prompt strings ourselves to be specific to ReadRight and Dolch words rather than generic. The final buildDolchStoryPrompt() in server.js has temperature: 0.7 (our choice for slight variation without going off-topic) and the word count constraint in both the system and user prompts to double-enforce it.
6. Testing / Verification	Sent 15 test requests through the proxy with different Dolch word lists and themes. Checked each response for story length (all were 3–5 sentences), presence of target words, and vocabulary complexity. Two responses used words slightly above kindergarten level, and we added 'Use simple vocabulary appropriate for the requested grade level' to the system prompt and retested until all outputs were consistently appropriate.

Entry 4: Understanding the AzureAssessor Blended Scoring Logic (Gemini)	
1. Context	While reviewing the inherited azure_assessor.dart code, I noticed it blends Azure's pronunciation accuracy score with a Levenshtein string similarity score (70% Azure + up to 30 points from similarity). I wanted to understand whether this blending approach was sound before deciding to keep it in our extended version, or whether it might produce misleading scores for children.
2. Prompt Excerpt	"This Flutter pronunciation assessment code blends Azure's accuracy score with a Levenshtein string similarity score: $\text{blendedAccuracy} = 0.7 * \text{rawAccuracy} + 30.0 * \text{sim}$. The similarity is computed between the reference word and what Azure recognized. Is this blending approach sound for a children's literacy app, or could it produce misleading scores in edge cases?"
3. AI Summary	Gemini identified two potential edge cases. First, if Azure mishears a completely wrong word that happens to be orthographically similar to the target (e.g., 'cat' vs 'bat'), the similarity boost could inflate the score unfairly. Second, for very short Dolch words like 'a' or 'I', the Levenshtein distance is always 0 or 1, so the similarity score is either 1.0 or 0.0 with no middle ground, and any recognition error on a one-letter word drops the similarity to 0 and removes the full 30-point boost abruptly.
4. Human Evaluation	The short-word edge case was the more relevant concern for our app since Dolch lists include many one- and two-letter words. We looked at the code ourselves and confirmed: <code>_stringSimilarity('a', 'the')</code> = 0.0 (length 1 vs 3, distance 3,

	maxLen 3, sim = 0), so the boost disappears entirely on a one-character mismatch. Gemini's observation was accurate. The 'cat'/'bat' example was less relevant because Azure's pronunciation assessment is phoneme-level, not just string-level.
5. Final Decision	Accepted Gemini's analysis of the short-word concern. We noted it in our UX critique in the PRD and flagged it as a known limitation to revisit in M3. We did not modify azure_assessor.dart for M2 since changing the scoring formula would be a non-trivial behavioral change that could break calibration. The blending logic is inherited code and works adequately for the spike demo.
6. Testing / Verification	Manually traced the _stringSimilarity and _levenshtein functions in azure_assessor.dart with one-letter inputs on paper. Confirmed: for input 'a' vs 't', levenshtein = 1, maxLen = 1, sim = (1 - 1/1) = 0.0. So blendedAccuracy = 0.7 * rawAccuracy + 0.0, meaning the similarity adds nothing for single-character mismatches. Gemini's analysis was confirmed by the code.

Entry 5: iOS Firebase Configuration - Running on Simulator (Claude)	
1. Context	When we tried to run the app on the iOS simulator, it crashed immediately with 'DefaultFirebaseOptions have not been configured for ios'. The firebase_options.dart file only had web, Android, and Windows configurations. We needed to add iOS support so the video demo could show the app on a mobile platform.
2. Prompt Excerpt	"Our Flutter app crashes on iOS with this error: DefaultFirebaseOptions have not been configured for ios. The firebase_options.dart was generated by FlutterFire CLI and has web and Android configured but iOS throws an UnsupportedError. What is the correct fix and what files does it change?"
3. AI Summary	Claude explained that the fix is to run flutterfire configure again from inside the readright_app directory, select the existing Firebase project (readright-fe68d), and ensure iOS is checked as a target platform. The CLI will regenerate firebase_options.dart with a proper iOS case and will place a GoogleService-Info.plist in ios/Runner/. It noted that this list file should be committed to the repo and is safe to do so since it contains only project identifiers, not secret keys.
4. Human Evaluation	The explanation was accurate. Running flutterfire configure did regenerate firebase_options.dart with an iOS FirebaseOptions block and placed GoogleService-Info.plist in ios/Runner/ automatically. The note about committing the list was important, and we had initially added it to .gitignore thinking it was a secret file, which broke the iOS build for teammates who cloned the repo.
5. Final Decision	Accepted. Ran flutterfire configure, selected the existing project, checked iOS. Confirmed firebase_options.dart now has a case TargetPlatform.iOS: return ios; instead of a throw. Removed GoogleService-Info.plist from .gitignore and

	committed it. The iOS simulator build now launches and reaches the login screen.
6. Testing / Verification	Ran flutter run on the iPhone 16 Pro Max simulator after the fix. The app launched past the Firebase initialization in main.dart and reached the login screen without crashing. Confirmed by checking firebase_options.dart that the iOS FirebaseOptions block has a valid appId and googleAppID field matching the Firebase console entry for the iOS app.

Entry 6: Confirming .env and backend/.env Are Both Gitignored (Claude)	
1. Context	The assignment's AI and cost guardrails explicitly require that API keys never be committed to GitHub. We have two .env files, one in readright_app/ for the Azure speech key and one in backend/ for the OpenAI key. I wanted to verify our .gitignore setup actually covers both before the M2 submission, since a leaked key would be a hard violation.
2. Prompt Excerpt	"We have two .env files in our Flutter + Node.js project: readright_app/.env for Azure keys and backend/.env for the OpenAI key. Our root .gitignore has a single line: .env. Does a .env entry in the root .gitignore cover both nested .env files, or do we need separate gitignore entries?"
3. AI Summary	Claude explained that a .env pattern in the root .gitignore does match .env files in subdirectories, and Git's gitignore patterns without a leading slash match the pattern anywhere in the repository tree. So a single .env line in the root .gitignore covers readright_app/.env, backend/.env, and any other .env file at any depth. It also recommended running git status and git ls-files to confirm the files are not currently tracked.
4. Human Evaluation	We verified this with git ls-files before trusting Claude's explanation. Neither readright_app/.env nor backend/.env appeared in the output of git ls-files, confirming they were not staged or tracked. We also ran git status with both files present and confirmed they showed as 'untracked' rather than 'modified' or 'new file'. Claude's explanation of glob pattern scope was correct.
5. Final Decision	Accepted. No changes needed, the single root .gitignore .env entry already covers both secret files. We added a comment to the root .gitignore documenting that this line intentionally covers both the Azure and OpenAI .env files, so future teammates do not add duplicate entries thinking one location is unprotected.
6. Testing / Verification	Ran git ls-files, grep .env, no output, confirming neither .env file is tracked. Ran git status in the repo root with both files present, both showed under 'Untracked files' with red text, not under 'Changes to be committed'. Also checked git log --all --full-history -- '**/.env' and confirmed no past commits had ever included either .env file.